

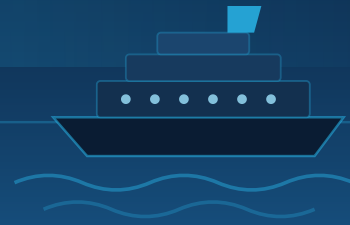
CARNIVAL

HOLLAND AMERICA



Casino Gaming Analytics

Snowflake AI Data Cloud · Hands-On Lab



Carnival Cruise Gaming HOL -- User 01

Welcome! This folder contains everything you need for the lab. All files are pre-configured for your Snowflake environment -- just follow the sections below in order.

Your Environment

Setting	Value
Role	<code>HOL_USER_01_ROLE</code>
Warehouse	<code>HOL_USER_01_WH</code>
Database	<code>HOL_USER_01_DB</code>

Where to Find Your Files

Your lab files are available in **two places** -- use whichever is easier:

Option A -- Snowflake Workspace (recommended):

1. In Snowsight, go to **Projects > Workspaces**
2. Open **HOL_WORKSPACES > USER_HOL_01**
3. Click any `.sql` file to open it directly as a worksheet, or the `.ipynb` file (Section 9) to open it as a notebook

Option B -- This GitHub folder:

1. Click any `.sql` file link below
2. Copy its contents into a new Snowsight SQL Worksheet

Sections 1-8 and 10 are SQL worksheets. **Section 9 is a notebook** and must be run from your Workspace -- it cannot be pasted into a worksheet.

These directions in other formats

This page is also provided as a formatted document, in case you would rather read it side-by-side with Snowsight or print it:

File	Use it for
README.html	Open in a browser -- same content, easier to read, links still work
README.pdf	Print it or read offline (US Letter)

Both are generated from this `README.md`, so it stays the single source of truth. If something looks out of date, trust the `.md` and the `.sql` files.

How This Works

For each section below:

1. Open the SQL file (from your workspace or from the link below)
2. **Run the statements top to bottom** (or highlight and run one section at a time)
3. Read the comments in the SQL -- they explain what each step does

Every SQL file starts with `USE ROLE / USE WAREHOUSE / USE DATABASE` so your context is always set correctly. You cannot break anyone else's work.

Section 1: Warehousing and Data Loading

Open: [01_warehousing_and_loading.sql](#)

What you will learn:

- Create a medallion architecture (BRONZE / SILVER / GOLD schemas)
- Load CSV data from a shared Snowflake stage into your own database
- Explore gaming data with basic SQL queries

Business context: Before we can analyze casino operations, we need to ingest the raw data from ship systems into Snowflake. The medallion architecture (bronze -> silver -> gold) keeps data organized as it moves from ingestion to business-ready.

What to do:

1. Paste the file into a new Worksheet
2. Run all statements (or run them one section at a time)
3. Verify you see row counts for all 6 tables at the end

Key queries to look at:

- Fleet overview (all 12 ships across both brands)
 - Revenue by game type (which games make the most money?)
 - Carnival vs Holland America comparison
-

Section 2: Automated Pipelines with Dynamic Tables

Open: [02_dynamic_tables.sql](#)

What you will learn:

- Create Dynamic Tables that automatically transform data
- Build a multi-layer pipeline: BRONZE -> SILVER -> GOLD
- Understand target lag and automatic refresh

Business context: Casino operations teams need real-time dashboards. Dynamic Tables automatically keep downstream aggregations in sync when raw data changes -- no scheduling, no stored procedures, no manual refreshes.

What to do:

1. Paste the file into a new Worksheet
2. Run all statements to create 4 Dynamic Tables
3. Run `SHOW DYNAMIC TABLES` to verify they are refreshing
4. Query the analytics layer to see aggregated results

Dynamic Tables you will create:

Layer	Table	What it does
SILVER	GAMING_TRANSACTIONS_H	Joins transactions with ship, game, player, and voyage details
SILVER	PLAYER_SUMMARY_H	Aggregates each player's total wagered, won, bet count, favorite game
GOLD	DAILY_SHIP_REVENUE	Daily revenue by ship and game type with house edge
GOLD	VOYAGE_PERFORMANCE	Per-voyage revenue, revenue per day, revenue per passenger

Section 3: Cortex AI Functions Sampler + Playground

Open: [03_cortex_playground.sql](#)

What you will learn:

- Run every major Cortex `AI_*` function against your own gaming data
- Understand what each one returns, and how to pull values out of it
- Compare LLM models side-by-side in Cortex Playground

Business context: Before deploying AI at scale, you want to know which function fits which job -- and experiment with prompts and models interactively.

What to do:

1. Run the file top to bottom. Each numbered block is one function, and the comments explain when to use it.
2. Then open **AI & ML > Cortex Playground** in Snowsight
3. Select two models to compare (e.g., `llama3.1-8b` and `mistral-large2`)
4. Copy a review from your query results and paste it into the Playground
5. Ask the model to summarize it -- compare the two outputs
6. Try changing the system prompt to: *"You are a cruise ship casino operations analyst. Provide concise, actionable summaries of customer feedback."*

Functions covered, and the gotcha for each:

Function	Returns	How to read it
<code>AI_COMPLETE(model, prompt)</code>	Text	Use directly
<code>AI_SENTIMENT(text [, categories])</code>	OBJECT	<code>:categories[0]:sentiment::VARCHAR</code> for overall; pass up to 10 categories for aspect-level sentiment
<code>AI_CLASSIFY(text, labels)</code>	OBJECT	<code>:labels[0]::VARCHAR</code> (note: <code>labels</code> , plural, and it is an array)
<code>AI_SUMMARIZE(text)</code>	Text	Use directly -- no model argument
<code>AI_EXTRACT(text, fields)</code>	OBJECT	<code>:response:<field_name>::VARCHAR</code>
<code>AI_TRANSLATE(text, from, to)</code>	Text	Use directly
<code>AI_FILTER(PROMPT('...{0}', col))</code>	BOOLEAN	Must wrap the question in <code>PROMPT()</code> -- the plain two-argument form is for images only

Note on sentiment: `AI_SENTIMENT` returns **labels**, not a -1 to +1 score. Values are `positive` , `negative` , `neutral` , `mixed` , and `unknown` . Aggregate it with `COUNT_IF` and percentages rather than `AVG()` .

Section 4: AI-Powered Analytics with SQL Functions

Open: [04_ai_sql_functions.sql](#)

What you will learn:

- Score review sentiment with `AI_SENTIMENT()`
- Classify reviews into business categories with `AI_CLASSIFY()`
- Summarize long reviews with `AI_SUMMARIZE()`
- Generate an executive briefing with `AI_COMPLETE()`

Business context: Casino managers receive thousands of player reviews. AI functions let you instantly process them all -- scoring sentiment, categorizing feedback, and generating summaries -- directly in SQL without external tools.

What to do:

1. Paste the file into a new Worksheet
2. Run the **sentiment analysis** query -- see which ships rank highest and lowest
3. Run the **classification** query -- understand what players talk about most (Dealer Quality? Game Variety? Atmosphere?)
4. Run the **executive briefing** generation -- AI writes a management summary for you
5. Create the `ENRICHED_REVIEWS` view for use in later sections

Cortex AI Functions you will use:

Function	What it does
<code>AI_SENTIMENT()</code>	Returns overall and aspect-level sentiment labels (<code>positive</code> , <code>negative</code> , <code>neutral</code> , <code>mixed</code> , <code>unknown</code>)
<code>AI_CLASSIFY()</code>	Categorizes text into labels you define (e.g., "Dealer Quality", "Wait Times")
<code>AI_SUMMARIZE()</code>	Creates a concise summary of longer text
<code>AI_COMPLETE()</code>	Generates new text from a prompt (e.g., management briefings)

Section 5: Semantic Search with Cortex Search

Open: [05_cortex_search.sql](#)

What you will learn:

- Materialize AI-enriched data, then index it with a Cortex Search Service
- Perform semantic (meaning-based) searches that go beyond keywords
- Filter search results by ship, brand, and sentiment

Business context: When investigating specific issues (e.g., "are dealers being rude on the Mardi Gras?"), keyword search misses relevant feedback that uses different phrasing. Cortex Search understands meaning, not just keywords.

What to do:

1. Paste the file into a new Worksheet
2. Run the `CREATE OR REPLACE TABLE GOLD.REVIEWS_FOR_SEARCH` statement first
3. Then run the `CREATE CORTEX SEARCH SERVICE` statement (takes ~1 minute to index)
4. Run the sample search queries and examine the results

5. Try your own searches -- think about what a casino operations manager would look for

Why two steps instead of one? It is tempting to call `AI_SENTIMENT` directly inside the search service definition. Do not. Cortex Search change tracking does not support inline AI function calls, and the service refreshes on its `TARGET_LAG` -- so you would also pay to re-score every review on every refresh. Materializing to a table scores each review exactly once.

Sample searches to try:

- `"slot machines paying out well"` -- finds reviews about winning, even without the word "payout"
- `"rude or unfriendly dealer experience"` -- surfaces complaints using various phrasings
- `"poker tournament experience on sea days"` -- finds tournament-related feedback
- `"best casino experience"` with a Carnival-only filter -- shows filtered semantic search
- `"payouts and odds"` filtered to `sentiment_category = 'negative'` -- combines semantic search with an AI-derived attribute

Section 6: Conversational BI with Cortex Analyst and CoWork

Open: [06_cortex_analyst_cowork.sql](#) and [semantic_model.yaml](#)

What you will learn:

- Create a semantic view from a YAML model that describes your gaming data
- Create a Cortex Agent **declaratively in SQL** (`CREATE AGENT`) with two tools
- Register your agent so it appears in Snowflake CoWork
- Ask natural language questions in Snowflake CoWork -- no SQL needed

Business context: Executives and managers should not need to write SQL. Cortex Analyst translates plain-English questions into SQL queries against your semantic model. CoWork provides the chat interface.

What to do:

1. Make sure you have finished **Section 5** -- the agent uses your Cortex Search service
2. Paste `06_cortex_analyst_cowork.sql` into a new Worksheet

3. Run it top to bottom. There is **nothing to upload by hand** -- the script:
 - creates a stage and file format
 - uses `COPY FILES` to pull `semantic_model.yaml` straight from your workspace
 - builds the semantic view with `SYSTEM$CREATE_SEMANTIC_VIEW_FROM_YAML`
 - creates your agent with `CREATE AGENT ... FROM SPECIFICATION`
4. **Register the agent in CoWork.** Creating an agent does *not* make it show up in CoWork by itself -- Step 6 of the script adds it to the account's CoWork list for you. That step is safe to re-run.
5. **Chat with it:** go to ai.snowflake.com (<https://ai.snowflake.com>) (or **AI & ML > Agents** in Snowsight) and pick **"Casino Analytics - User 01"**. If it is not listed, re-run Step 6 and refresh the page.
6. **Start asking questions!**

Why the extra step? This account uses a *CoWork object*, which holds a curated list of the agents CoWork displays. When that object exists, an agent is only shown once it has been added to it. You can only add agents you own, so you will only ever see your own agent in the list.

Your agent has three tools, and picks between them automatically:

Tool	Type	Used for
Casino_Metrics	Cortex Analyst	Numbers: revenue, wagers, house edge, players
Review_Search	Cortex Search	Opinions: what guests said, complaints
data_to_chart	Charting	Turning results into visualizations

Sample questions to try in CoWork:

Numbers (routes to `Casino_Metrics`):

- "What was total gaming revenue by ship?"
- "Which game type generates the highest house edge?"
- "Compare Carnival vs Holland America gaming revenue"
- "Show me the top 10 players by total wagered"
- "How does revenue vary by player loyalty tier?"

Opinions (routes to `Review_Search`):

- "What are guests complaining about in the casino?"
- "What do guests say about the dealers?"
- "Are there complaints about smoke or ventilation?"

Both tools at once:

- "Which ship has the lowest revenue, and what are guests saying about it?"
 - "Chart revenue by brand and summarize guest sentiment for each"
-

Section 7: Data Governance

Open: [07_governance.sql](#)

What you will learn:

- Tag PII columns (names, email, phone) with object tags
- Create dynamic data masking policies to protect sensitive data
- Create row access policies to restrict data by brand
- See how governance controls are transparent and role-based

Business context: Player data contains PII (names, emails, phone numbers) that must be protected.

Governance controls ensure analysts can query gaming patterns without seeing raw personal data, and brand-specific teams only see their own ships.

What to do:

1. Paste the file into a new Worksheet
2. Create tags and apply them to the PLAYERS table columns
3. Create and apply masking policies for email, phone, and names
4. Query the PLAYERS table -- you should see **unmasked** data because your role is authorized
5. Create and apply the row access policy on SHIPS
6. Run `SHOW TAGS` , `SHOW MASKING POLICIES` , `SHOW ROW ACCESS POLICIES` to see your governance posture

What you will create:

Object	Type	What it does
PII_TYPE	Tag	Classifies columns as NAME, EMAIL, or PHONE
SENSITIVITY_LEVEL	Tag	Classifies tables as PUBLIC, INTERNAL, or CONFIDENTIAL
EMAIL_MASK	Masking Policy	Hides email prefix for unauthorized roles (shows <code>***@domain.com</code>)
PHONE_MASK	Masking Policy	Shows only last 4 digits for unauthorized roles
NAME_MASK	Masking Policy	Shows only first initial for unauthorized roles
BRAND_ROW_POLICY	Row Access Policy	Filters ship data by brand based on the querying role

Section 8: Data Collaboration via Marketplace

Open: [08_collaboration.sql](#)

What you will learn:

- Enrich your gaming data with external weather and port data
- Analyze how weather and sea days affect casino revenue
- Understand zero-copy data sharing concepts

Business context: Do passengers gamble more on sea days when they cannot leave the ship? Does rough weather keep passengers indoors at the casino? External data enrichment answers these questions without building ETL pipelines.

What to do:

1. Paste the file into a new Worksheet
 2. Create the simulated port weather data (in a real scenario, this comes from the Snowflake Marketplace)
 3. Join weather data with your voyage performance metrics
 4. Look at the correlation between sea days and gaming revenue
 5. Compare revenue on rough-sea vs calm-sea voyages
-

Section 9: Python and Snowpark

Open: [09_snowpark_notebook.ipynb](#)

This one is a **notebook**, not a worksheet. Open it directly from your Workspace file list and run the cells top to bottom.

What you will learn:

- How Snowpark DataFrames are **lazy** -- they build SQL, they do not move data
- `select` / `filter` / `join` / `group_by` / `agg` against your gaming tables
- Window functions to rank players within each brand
- Writing results back into a Snowflake table with `save_as_table`
- Registering a **Python UDF** and calling it like any SQL function
- Calling Cortex AI functions from Snowpark
- Charting the results with matplotlib

Business context: Worksheets are great for exploration, but production pipelines, applications, and scheduled jobs are usually written in Python. This section rebuilds the lab's analytics in Snowpark so you can see that the API is just SQL with a Python face -- the compute still happens in Snowflake and the data never moves.

What to do:

1. Open the notebook from your Workspace
2. Pick `H0L_USER_01_WH` as the warehouse in the notebook toolbar
3. Run the cells in order, reading the markdown between them
4. If `import matplotlib` fails, add it from the **Packages** menu at the top

Prerequisite: Sections 1 and 2 must be complete -- the notebook reads your BRONZE tables and writes to `GOLD.PLAYER_VALUE_SNOWPARK` .

Section 10: Cleanup (Optional)

Open: [10_cleanup.sql](#)

Every statement in this file is commented out on purpose, so opening and running it by accident cannot destroy your work.

Run it **only when your lab facilitator says the lab is over**. To use it, uncomment the statements (select the lines and press `Cmd+/` or `Ctrl+/`), then run the script. It drops all your schemas and suspends your warehouse.

Troubleshooting

Problem	Fix
"Object does not exist"	Make sure you completed Section 1 first -- it creates all schemas and loads data
"Insufficient privileges"	Run <code>SELECT CURRENT_ROLE();</code> -- you should see <code>H0L_USER_01_ROLE</code> . If not, run <code>USE ROLE H0L_USER_01_ROLE;</code>
Dynamic Table not refreshing	Run <code>SHOW DYNAMIC TABLES IN DATABASE H0L_USER_01_DB;</code> and check the <code>SCHEDULING_STATE</code> column
Cortex Search taking a long time	The first indexing takes 1-2 minutes. Subsequent queries are fast.
CoWork not finding answers	Verify <code>COPY FILES</code> pulled <code>semantic_model.yaml</code> onto the stage (<code>LIST @GOLD.SEMANTIC_MODELS;</code>) and the semantic view was created successfully
My agent is not listed in CoWork	Creating an agent does not publish it. Re-run Step 6 of Section 6 (<code>ALTER SNOWFLAKE INTELLIGENCE ... ADD AGENT</code>), then refresh ai.snowflake.com (https://ai.snowflake.com). Confirm with <code>SHOW AGENTS IN SNOWFLAKE INTELLIGENCE SNOWFLAKE_INTELLIGENCE_OBJECT_DEFAULT;</code>
AI function errors	Ask your instructor to verify the <code>SNOWFLAKE.CORTEX_USER</code> database role is granted to <code>H0L_USER_01_ROLE</code>
An AI column comes back all NULL (no error)	You are almost certainly using the wrong JSON accessor. These fail <i>silently</i> -- see the table below.
Wrong results in queries	Check that you are in the right database: <code>SELECT CURRENT_DATABASE();</code> should show <code>H0L_USER_01_DB</code>

Silent NULL traps with AI functions

These do not raise an error -- they just return NULL for every row, which is easy to mistake for "the AI found nothing." If a column is entirely NULL, check this first:

Function	Wrong (silent NULL)	Correct
AI_CLASSIFY	:label	:labels[0] -- it is an array , and the key is plural
AI_EXTRACT	:my_field	:response:my_field -- fields are nested under response
AI_SENTIMENT	:sentiment	:categories[0]:sentiment

To debug any of these, select the raw object first and look at its actual shape:

```
SELECT AI_CLASSIFY(review_text, ['Dealer Quality','Pricing']) AS raw_object
FROM BRONZE.PLAYER_REVIEWS LIMIT 1;
```